

République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Higher Institute of Sciences



Projet de fin d'étude

En : Mathématiques et Informatique

Spécialité : Ingénierie de Systèmes d'Information & Logiciels

Intitulé :

Vectorization Automatique d'Images Matricielles par Deep Learning

Encadré par :

ABDELLAHOUM Hamza

Présenté par :

BITAM Amir

AITHAMOUDI Zakaria

Membres du Jury :

????

????

Soutenu le :

??/07/2026

Année universitaire : 2025/2026

Remerciements

Nous tenons à exprimer notre profonde gratitude envers Allah, le Tout-Puissant, pour nous avoir accordé la force, la patience et la persévérance nécessaires pour mener à bien ce projet de fin d'étude.

Nous remercions sincèrement notre encadreur **ABDELLAHOU** **Hamza** pour sa précieuse guidance, son soutien constant et ses précieux conseils tout au long de ce projet.

Nos vifs remerciements s'adressent également aux membres du jury **????** et **????** pour avoir accepté d'évaluer notre travail.

Dédicace

Résumé

Nous proposons dans le cadre de notre projet de fin d'études un modèle d'IA qui traite le zoom et le dézoom dans les images numériques. En zoomant sur une image de taille donnée, sa qualité est souvent altérée et l'image peut devenir pixélisée. Notre travail consiste à améliorer ces images en augmentant leur qualité.

En ce qui concerne le dézoom, en réduisant les dimensions de l'image, il faut minimiser le nombre de pixels tout en préservant l'esthétique de l'image.

Nous avons créé un modèle CNN avec une architecture adaptée, piloté par des métaheuristiques (algorithme génétique) permettant de sélectionner les paramètres optimaux afin d'obtenir de bonnes performances avec un nombre minimal de paramètres.

Mots-clés : Redimensionnement d'images, Super Résolution, Réseaux de Neurones Convolutifs (CNN), Algorithme Génétique, Métaheuristiques, PSNR, SSIM.

Abstract

Keywords : Image Resizing, Super Resolution, Convolutional Neural Networks (CNN), Genetic Algorithm, Metaheuristics, PSNR, SSIM.

Liste des Acronymes

XML	eXtensible Markup Language
SVG	Scalable Vector Graphics
GA	Genetic Algorithm
CNN	Convolutional Neural Networks
PSNR	Peak Signal to Noise Ratio
HR	High Resolution
LR	Low Resolution
SSIM	Structural SIMilarity
SRCNN	Super Resolution Convolutional Neural Network
SRGAN	Super Resolution Generative Adversarial Network
MAE	Mean Absolute Error
MSE	Mean Squared Error
ReLU	Rectified Linear Unit
ResNet	Residual Network
SE	Squeeze-and-Excitation
CAR	Content Adaptive Resampler
FARF	Feature Augmented Random Forest
DRDN	Deep Residual Dense Network
EDSAN	Enhanced Dense Space Attention Network
PPI	Pixels Per Inch
ViT	Vision Transformer
UDF	Unsigned Distance guided Focal loss
SLIC	Simple Linear Iterative Clustering
DINO	self-Distillation with NO labels
AMP	Automatic Mixed Precision

Table des matières

Remerciements	i
Dédicace	ii
Résumé	iii
Abstract	iv
Liste des Acronymes	v
Introduction Générale	2
1 État de l'Art	3
1.1 Introduction	3
1.2 La vectorisation d'images	3
1.2.1 Images matricielles	3
A. Pixels	3
1.2.2 Images vectorielles	4
A. Courbes de Bézier	5
B. Format SVG	5
1.3 Apprentissage automatique et apprentissage profond	5
1.3.1 Apprentissage automatique et apprentissage auto-supervisé . . .	6
1.3.2 Apprentissage profond	6
A. Réseaux de neurones	7
B. Transformers	7
C. Vision Transformers	8
1.4 Mécanismes d'attention	9
1.4.1 Self-attention	10
1.4.2 Cross-attention	11
1.5 Concepts utilisés dans la vectorisation par apprentissage profond	11
1.5.1 Superpixels	12
1.5.2 Rendu différentiable	12
1.5.3 Fonctions de perte	12
1.5.4 Apprentissage coarse-to-fine	13
1.6 Évaluation des résultats	13
1.6.1 MSE	13
1.6.2 PSNR	14

1.6.3	SSIM	14
1.6.4	LPIPS	14
1.6.5	Temps d'inférence	15
1.7	Travaux connexes	15
1.7.1	Approches algorithmiques	15
A.	Potrace	15
1.7.2	Approches basées sur l'optimisation	16
A.	DiffVG	16
B.	LIVE	16
1.7.3	Approches basées sur l'apprentissage profond	16
A.	Im2Vec	17
B.	SuperSVG	17
1.8	Synthèse comparative des approches	18
1.9	Conclusion	19
2	Approche proposée	20
2.1	Introduction	20
2.2	Vue d'ensemble de la méthode	20
2.2.1	Principe général	21
2.2.2	Choix de conception	21
2.3	Préparation des données	22
2.3.1	Jeu de données utilisé	23
2.3.2	Décomposition en superpixels	23
2.3.3	Recadrage et construction de l'entrée	23
2.4	Architecture du modèle	24
2.4.1	Extraction des caractéristiques avec DINOv3	24
2.4.2	Requêtes apprenables et cross-attention	25
2.4.3	Interactions entre les chemins	26
2.4.4	Représentation d'un chemin vectoriel	26
2.5	Rendu différentiable et fonctions de perte	27
2.5.1	Construction de la scène DiffVG	28
2.5.2	Perte de reconstruction normalisée	28
2.5.3	Pénalité de débordement	29
2.5.4	Objectif global	29
2.6	Stratégie d'entraînement	30
2.6.1	Optimisation des paramètres	30
2.6.2	Sauvegarde et reprise	31
2.7	Inférence et génération du SVG	31
2.7.1	Traitement de l'image complète	32

2.7.2	Algorithme d'inférence	32
2.7.3	Différences entre entraînement et inférence	33
2.7.4	Périmètre de la version implémentée	34
2.8	Conclusion	35
3	Expérimentations et Résultats	36
	Conclusion Générale	37
	Bibliographie	38

Table des figures

1.1	Comparaison entre une image matricielle (a) et une image vectorielle (b).	4
1.2	Exemple du contenu d'un fichier .svg	5
1.3	Structure simplifiée d'un encodeur Transformer	8
1.4	Architecture générale d'un Vision Transformer	8
1.5	Attention par produit scalaire mise à l'échelle	10
2.1	Pipeline générale de l'approche proposée, de l'image matricielle aux chemins SVG.	21
2.2	Chemin fermé composé de quatre courbes de Bézier cubiques et de douze points prédits.	27

Liste des tableaux

1.1	Synthèse comparative des principales approches de vectorisation étudiées.	18
2.1	Principaux modules de la pipeline proposée.	22
2.2	Configuration enregistrée pour l'entraînement du modèle à 128 chemins par superpixel.	31
2.3	Comparaison du flux d'entraînement et du flux d'inférence.	34

Introduction Générale

Le traitement d'images est un domaine de recherche important qui trouve des applications dans divers domaines tels que la médecine, la surveillance, la sécurité, la robotique, l'automobile, etc.

L'un des problèmes les plus courants dans ce domaine est la qualité de l'image qui peut être altérée lorsqu'elle est agrandie ou réduite en taille.

Ce mémoire est organisé comme suit :

- **Chapitre 1** : État de l'art — notions fondamentales et principales approches de vectorisation d'images.
- **Chapitre 2** : Approche proposée — préparation par superpixels, architecture DINOv3 avec attention, rendu différentiable et génération du SVG.
- **Chapitre 3** : Expérimentations et Résultats — évaluation quantitative et qualitative avec comparaisons.

Chapitre 1

État de l'Art

1.1 Introduction

Dans ce chapitre, nous exposerons d'abord les concepts clés à la vectorisation d'images, à l'apprentissage automatique, et les architectures utilisées tout au long de ce travail, ceci nous permettra de mieux appréhender ensuite les travaux effectués sur le sujet.

1.2 La vectorisation d'images

Dans cette section nous tenterons de définir les termes et concepts liés à la transformation d'images matricielles en images vectorielles, dite, vectorisation d'images. En soulignant notamment la différence entre les images dites matricielles et vectorielles. En opposant ainsi les deux types d'images nous pourrions mieux cerner la motivation derrière ce travail et pourquoi la vectorisation d'image constitue un exercice complexe de la vision par ordinateur.

1.2.1 Images matricielles

Les images matricielles peuvent être définies comme des fonctions bidimensionnelles $f(x, y)$, où x et y représentent des coordonnées spatiales planes et où la valeur de f associée à chaque paire de coordonnées correspond à l'intensité ou à la couleur du point considéré. Plus simplement, elles peuvent être représentées comme une grille de pixels organisée selon une largeur et une hauteur données [1]. Les images matricielles présentent toutefois une limite lors de leur agrandissement. En effet, comme elles sont constituées d'un nombre fixe de pixels, une augmentation importante de leurs dimensions peut rendre ces derniers visibles et produire un effet de pixellisation. La qualité de l'image se dégrade alors progressivement à mesure que le facteur d'agrandissement augmente.

A. Pixels

Un pixel, contraction de *picture element*, représente l'unité élémentaire d'une image numérique. Il correspond à un point de la grille matricielle et possède une position, définie par ses coordonnées spatiales, ainsi qu'une valeur décrivant son intensité ou sa couleur. Dans une image en niveaux de gris, cette valeur correspond généralement à une intensité

lumineuse, tandis que dans une image couleur elle est souvent représentée par plusieurs composantes, par exemple rouge, vert et bleu. Ainsi, une image matricielle peut être comprise comme un ensemble fini de pixels organisés selon une largeur et une hauteur données [1]. Cette représentation rend le contenu directement dépendant de la résolution : lorsqu'une image est fortement agrandie, les pixels deviennent progressivement visibles, ce qui produit l'effet de pixellisation. Ainsi, le pixel constitue l'unité élémentaire de l'image matricielle, mais il impose également une granularité fixe qui limite la conservation des détails lors des changements d'échelle.

1.2.2 Images vectorielles

Les images vectorielles sont décrites par un ensemble de formes géométriques individuelles, telles que des cercles, des segments de droite ou des courbes de Bézier. Chacune de ces formes est enregistrée avec les paramètres mathématiques et les attributs qui la définissent, notamment ses points d'ancrage, sa couleur de remplissage ou l'épaisseur de son contour. Contrairement aux images matricielles, dont le nombre de pixels est fixe, les images vectorielles peuvent être agrandies sans perte de qualité, car leur rendu est recalculé pour chaque résolution [2]. Comme l'illustre la figure 1.1, l'image matricielle présentée en (a) devient pixellisée lors de son agrandissement, tandis que l'image vectorielle présentée en (b) conserve des contours nets grâce à sa définition mathématique. Les images vectorielles sont donc largement utilisées dans les domaines du graphisme et de la conception. Cependant, la transformation d'une image matricielle en une image vectorielle reste complexe en raison de la différence entre leurs modes de représentation.

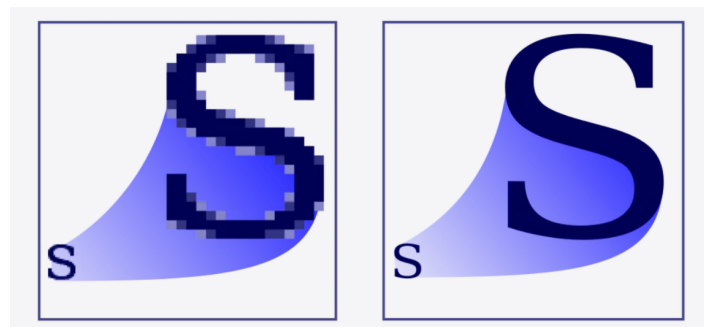


FIGURE 1.1 – Comparaison entre une image matricielle (a) et une image vectorielle (b).

Afin de mieux comprendre la manière dont une image vectorielle est construite et enregistrée, il convient de présenter deux éléments essentiels. Les courbes de Bézier permettent de représenter des contours et des formes complexes à partir d'un nombre limité de points de contrôle, tandis que le format SVG fournit une structure permettant de décrire et de stocker ces primitives vectorielles.

A. Courbes de Bézier

Une courbe de Bézier est définie comme une fonction polynomiale dont la forme est dictée par la position de points de contrôle. La courbe passe par le premier et le dernier point (points d'ancrage), tandis que les points intermédiaires agissent comme des "aimants" qui tirent la courbe vers eux [2].

Les courbes de Bézier font partie des formes géométriques individuelles qui composent les images vectorielles et se distinguent par le faible nombre de données numériques nécessaires à la représentation de formes complexes.

B. Format SVG

SVG (Scalable Vector Graphics) est un format basé sur XML comportant des directives pour la définition de graphiques 2D[3]. Stockées dans des fichiers textes rendant les SVG compressible et indexable, ces directives SVG supportent aussi bien les formes géométriques vectorielle que les images et le texte. Pour les raisons précédentes ce sera ce format qui sera préféré pour la sortie du modèle.

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 1.1//EN" "http://
www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd">
<svg width="391" height="391" viewBox="-70.5 -70.5 391 391"
xmlns="http://www.w3.org/2000/svg" xmlns:xlink="http://
www.w3.org/1999/xlink">
<rect fill="#fff" stroke="#000" x="-70" y="-70" width="390"
height="390"/>
<g opacity="0.8">
  <rect x="25" y="25" width="200" height="200" fill="lime"
stroke-width="4" stroke="pink" />
  <circle cx="125" cy="125" r="75" fill="orange" />
  <polyline points="50,150 50,200 200,200 200,100" stroke="red"
stroke-width="4" fill="none" />
  <line x1="50" y1="50" x2="200" y2="200" stroke="blue" stroke-
width="4" />
</g>
</svg>
```

FIGURE 1.2 – Exemple du contenu d'un fichier .svg

1.3 Apprentissage automatique et apprentissage profond

Cette section présente les notions d'apprentissage automatique et d'apprentissage profond nécessaires à la compréhension des approches modernes de vectorisation d'images. Elle introduit d'abord les principaux modes d'apprentissage, notamment l'apprentissage auto-supervisé, puis décrit les architectures profondes utilisées pour extraire et traiter les caractéristiques visuelles, en particulier les réseaux de neurones, les Transformers et les Vision Transformers.

1.3.1 Apprentissage automatique et apprentissage auto-supervisé

L'apprentissage automatique est une discipline du domaine de l'intelligence artificielle dans laquelle, contrairement à l'approche programmatique classique, la machine ne reçoit pas une suite d'instructions détaillées pour résoudre une tâche donnée, car ces instructions seraient souvent trop complexes à définir explicitement [4]. Le principe consiste plutôt à apprendre à partir d'un ensemble de données afin d'améliorer progressivement une performance P sur une tâche T à partir d'une expérience E , selon la définition proposée par Mitchell [5].

Selon la nature des données et du signal de supervision disponibles, plusieurs modes d'apprentissage peuvent être distingués. Dans l'apprentissage supervisé, chaque exemple d'entrée est associé à une cible ou à une étiquette attendue. À l'inverse, dans l'apprentissage non supervisé, le modèle cherche à découvrir des structures présentes dans les données, telles que des groupes d'exemples similaires ou des représentations de plus faible dimension [6].

Cependant, dans le contexte de la vision par ordinateur moderne, ces deux catégories ne suffisent pas toujours à expliquer l'intérêt des modèles pré-entraînés utilisés pour l'extraction de caractéristiques visuelles. En effet, l'annotation manuelle de grands ensembles d'images est une tâche coûteuse et parfois difficile à généraliser à tous les domaines. C'est dans ce cadre que l'apprentissage auto-supervisé prend une place importante. Cette approche exploite des données non étiquetées tout en générant automatiquement un signal de supervision à partir des données elles-mêmes, par exemple en demandant au modèle de prédire une partie manquante, une transformation appliquée ou une relation entre différentes vues d'une même image [7]. Elle permet ainsi d'apprendre des représentations visuelles générales à partir de grandes collections d'images sans annotations humaines explicites, représentations qui peuvent ensuite être transférées vers d'autres tâches de vision par ordinateur [4, 7]. Dans le cadre de la vectorisation d'images, cet aspect est particulièrement pertinent, car l'objectif n'est pas seulement de classer une image, mais d'en extraire une représentation riche de sa structure, de ses régions et de ses détails visuels, afin de faciliter ensuite la prédiction de primitives vectorielles.

1.3.2 Apprentissage profond

L'apprentissage profond est une sous-branche de l'apprentissage automatique fondée principalement sur les réseaux de neurones profonds. Ces modèles sont constitués de plusieurs couches de neurones artificiels capables d'apprendre progressivement des représentations de plus en plus abstraites à partir des données. Cette approche s'est fortement développée grâce à l'augmentation de la puissance de calcul, à la disponibilité de grandes quantités de données et aux progrès réalisés dans l'entraînement des

réseaux profonds [4]. Dans le domaine du traitement d'images, les réseaux de neurones convolutionnels (CNN) ont longtemps constitué l'une des architectures dominantes, grâce à leur capacité à exploiter la structure spatiale locale des images et à extraire automatiquement des caractéristiques visuelles telles que les contours, les formes ou les textures [8, 1]. Cependant, les architectures récentes fondées sur l'attention, en particulier les Transformers et les Vision Transformers, ont progressivement élargi cette approche en permettant de modéliser des relations plus globales entre les différentes régions d'une image.

A. Réseaux de neurones

Les réseaux de neurones artificiels sont des modèles d'apprentissage automatique inspirés, de manière simplifiée, du fonctionnement des neurones biologiques. Ils sont constitués d'unités de calcul appelées neurones artificiels, organisées en couches et reliées entre elles par des connexions pondérées. L'apprentissage consiste à ajuster ces poids à partir des données d'entraînement afin que le réseau puisse produire une sortie correcte pour une tâche donnée [4]. Dans le domaine du traitement d'images, ces modèles sont utilisés pour apprendre automatiquement des fonctions de décision à partir d'exemples, notamment dans les problèmes de classification, de reconnaissance de formes et de reconstruction [1].

B. Transformers

Les Transformers sont une architecture de réseaux de neurones introduite initialement pour le traitement des séquences, notamment en traduction automatique. Contrairement aux réseaux récurrents, ils ne traitent pas les éléments de la séquence de manière strictement séquentielle, mais s'appuient principalement sur des mécanismes d'attention, en particulier l'auto-attention, afin de modéliser les relations entre les différents éléments d'une entrée [9]. Grâce à leur capacité à apprendre des dépendances complexes et à être entraînés efficacement sur de grandes quantités de données, les Transformers ont ensuite été largement utilisés dans de nombreux domaines de l'apprentissage profond, notamment le traitement du langage naturel, la vision par ordinateur et les modèles multimodaux [4]. Comme l'illustre la figure 1.3, un encodeur Transformer peut être représenté de manière simplifiée par un mécanisme d'auto-attention multi-têtes suivi d'un réseau feed-forward.

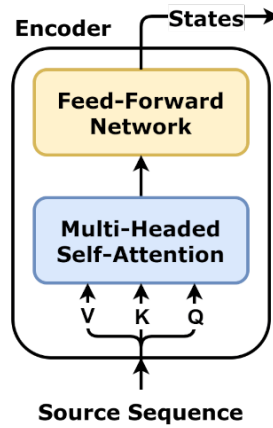


FIGURE 1.3 – Structure simplifiée d'un encodeur Transformer. Figure adaptée de dvgodoy, sous licence CC BY 4.0.

C. Vision Transformers

Les Vision Transformers, ou ViT, sont une adaptation de l'architecture Transformer au traitement des images. Au lieu de s'appuyer principalement sur des convolutions comme les CNN, une image est divisée en petits blocs appelés patches, par exemple de taille 16×16 . Ces patches sont ensuite transformés en vecteurs et traités comme une séquence d'éléments, de manière similaire aux tokens dans le traitement du langage naturel [10]. Grâce au mécanisme d'auto-attention, le modèle peut apprendre des relations globales entre différentes régions de l'image. Cette capacité rend les Vision Transformers particulièrement intéressants pour les tâches où la compréhension de la structure globale de l'image est importante, en complément des informations locales. Comme l'illustre la figure 1.4, les représentations des patches sont combinées à un encodage positionnel, puis transmises à un encodeur Transformer afin de produire la sortie du modèle.

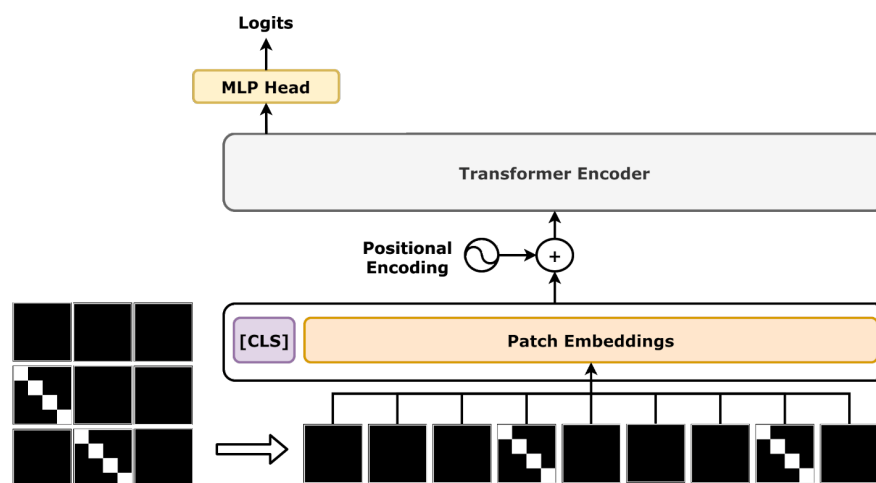


FIGURE 1.4 – Architecture générale d'un Vision Transformer. Figure adaptée de dvgodoy, sous licence CC BY 4.0.

1.4 Mécanismes d'attention

Les mécanismes d'attention constituent une composante importante des architectures modernes d'apprentissage profond, en particulier dans les modèles de type Transformer. Leur objectif est de permettre au modèle d'accorder une importance différente aux éléments d'une entrée selon leur pertinence pour la tâche à résoudre, au lieu de traiter toutes les informations de manière uniforme. Ce fonctionnement repose généralement sur trois éléments principaux : une requête, des clés et des valeurs. La requête représente l'information recherchée, les clés servent à mesurer la similarité avec cette requête, et les valeurs contiennent les informations à combiner [9].

Dans le cas de l'attention par produit scalaire mise à l'échelle, les requêtes, les clés et les valeurs sont regroupées respectivement dans les matrices Q , K et V . Les scores d'attention sont obtenus en calculant le produit entre les requêtes et les clés, puis en le normalisant par $\sqrt{d_k}$, où d_k désigne la dimension des clés. Un masque peut éventuellement être appliqué, notamment lorsque l'on souhaite empêcher le modèle d'accéder à certaines positions. La fonction softmax transforme ensuite, pour chaque requête, les scores obtenus en poids positifs compris entre 0 et 1, dont la somme est égale à 1. Les scores les plus élevés reçoivent ainsi les poids les plus importants, ce qui permet au modèle d'accorder davantage d'attention aux éléments considérés comme les plus pertinents. Ces poids sont finalement utilisés pour effectuer une combinaison pondérée des valeurs [9, 4]. Comme l'illustre la figure 1.5, ce mécanisme enchaîne le produit scalaire entre Q et K , la mise à l'échelle, l'application éventuelle d'un masque, la fonction softmax et la combinaison pondérée avec V .

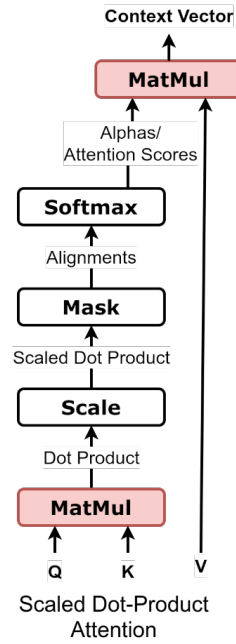


FIGURE 1.5 – Attention par produit scalaire mise à l'échelle. Figure adaptée de dvigodoy, sous licence CC BY 4.0.

Ce mécanisme peut être formalisé par l'équation suivante [9] :

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (1.1)$$

Cette équation montre que les similarités entre les requêtes Q et les clés K sont transformées en poids d'attention, puis utilisées pour pondérer les valeurs V . Le modèle peut ainsi privilégier les informations les plus pertinentes selon le contexte, y compris lorsqu'elles sont éloignées dans la séquence ou dans l'image [9, 4].

1.4.1 Self-attention

La self-attention, ou auto-attention, est un cas particulier du mécanisme d'attention dans lequel les requêtes, les clés et les valeurs proviennent de la même séquence. Elle permet donc à chaque élément d'une entrée de prendre en compte les autres éléments de cette même entrée afin de construire une représentation plus riche et contextualisée. Dans une phrase, par exemple, chaque mot peut être relié aux autres mots pour mieux comprendre son sens selon le contexte. Dans une image représentée sous forme de patches, chaque patch peut de la même manière interagir avec les autres régions de l'image [9]. Cette capacité à modéliser les relations internes d'une séquence explique l'importance de la self-attention dans les architectures Transformer. Dans le traitement d'images, cette propriété permet notamment de mettre en relation des régions éloignées d'une même image, contrairement aux opérations strictement locales qui se limitent à un voisinage restreint.

1.4.2 Cross-attention

La cross-attention, ou attention croisée, est une variante du mécanisme d'attention dans laquelle les requêtes ne proviennent pas de la même source que les clés et les valeurs. Contrairement à la self-attention, où un élément interagit avec les autres éléments d'une même séquence, la cross-attention permet à une représentation de s'appuyer sur une autre représentation externe.

Dans une architecture Transformer de type encodeur-décodeur, l'encodeur reçoit les données d'entrée et les transforme en représentations internes contenant les informations importantes de cette entrée. Le décodeur utilise ensuite ces représentations afin de générer progressivement la sortie attendue. Lors de la cross-attention, le décodeur produit les requêtes, tandis que les clés et les valeurs proviennent des représentations calculées par l'encodeur. Ce mécanisme permet ainsi au décodeur de sélectionner, à chaque étape de la génération, les informations de l'entrée les plus pertinentes [9, 4].

Dans le contexte de la vectorisation d'images, l'encodeur peut extraire les caractéristiques visuelles de l'image matricielle, tandis que le décodeur génère les paramètres des primitives vectorielles. La cross-attention permet alors aux requêtes associées aux primitives à produire d'interagir avec les clés et les valeurs issues de l'image encodée. Les paramètres vectoriels prédits sont ainsi guidés par les informations visuelles les plus pertinentes, ce qui facilite la génération de primitives adaptées au contenu de l'entrée.

1.5 Concepts utilisés dans la vectorisation par apprentissage profond

La vectorisation d'images par apprentissage profond repose sur plusieurs concepts techniques permettant de passer d'une image matricielle, composée de pixels, à une représentation vectorielle décrite par des formes paramétriques. Contrairement aux méthodes classiques de traçage, les approches récentes cherchent à apprendre ou à optimiser automatiquement les paramètres des primitives vectorielles afin de reconstruire au mieux l'image d'entrée. Pour cela, elles utilisent notamment la décomposition de l'image en régions plus simples, le rendu différentiable pour relier les paramètres vectoriels à leur rendu matriciel, ainsi que des fonctions de perte permettant de mesurer l'écart entre l'image originale et l'image reconstruite. Certaines méthodes adoptent également une stratégie progressive de type *coarse-to-fine*, dans laquelle la structure globale de l'image est d'abord reconstruite avant d'ajouter les détails fins [11, 12].

1.5.1 Superpixels

Les superpixels sont des régions obtenues en regroupant des pixels voisins présentant des caractéristiques similaires, notamment en termes de couleur et de position spatiale. Ils permettent de remplacer une représentation pixel par pixel par une représentation plus compacte, composée de régions visuellement cohérentes. Parmi les méthodes les plus connues, l'algorithme SLIC (*Simple Linear Iterative Clustering*) adapte le principe du k-means afin de générer efficacement des superpixels. Cette méthode est appréciée pour sa simplicité, sa rapidité, sa faible consommation mémoire et sa capacité à produire des régions qui respectent bien les contours de l'image [13]. Dans le cadre de la vectorisation d'images, les superpixels sont utiles car ils divisent une image complexe en sous-régions plus simples à traiter. SuperSVG exploite précisément cette idée en décomposant l'image d'entrée en superpixels contenant des pixels de couleurs et de textures similaires, puis en vectorisant ces régions de manière progressive [12].

1.5.2 Rendu différentiable

Le rendu différentiable est une technique qui permet de relier une représentation vectorielle à son rendu matriciel tout en conservant la possibilité de calculer des gradients. Dans le contexte de la vectorisation d'images, les paramètres des primitives vectorielles, comme les positions des points de contrôle, les couleurs ou l'épaisseur des traits, sont d'abord rendus sous forme d'image raster. L'image obtenue est ensuite comparée à l'image cible à l'aide d'une fonction de perte, puis les gradients de cette perte sont rétropropagés vers les paramètres vectoriels afin de les optimiser [11]. Cette approche permet donc d'utiliser des méthodes d'optimisation ou d'apprentissage profond pour ajuster automatiquement les formes vectorielles à partir d'une supervision dans le domaine des pixels. Elle est notamment utilisée dans des méthodes comme LIVE, qui optimise progressivement des chemins de Bézier dans un cadre de rendu entièrement différentiable afin de produire des SVG organisés en couches [14].

1.5.3 Fonctions de perte

Les fonctions de perte jouent un rôle central dans la vectorisation d'images par apprentissage profond, car elles permettent de mesurer l'écart entre l'image matricielle d'origine et l'image reconstruite à partir des primitives vectorielles. Dans les approches utilisant un rendu différentiable, les chemins SVG prédits ou optimisés sont d'abord rasterisés, puis comparés à l'image cible à l'aide d'une fonction de perte calculée dans l'espace des pixels. Une perte simple, comme la MSE, peut être utilisée pour minimiser la différence entre les deux images, mais elle ne suffit pas toujours à préserver correctement la structure, les contours ou la couleur des objets. Par exemple, LIVE montre qu'une

optimisation fondée uniquement sur la MSE peut conduire à un effet de couleur moyenne, puis propose une perte UDF afin de donner plus d'importance aux pixels proches des contours, ainsi qu'une perte Xing pour limiter les auto-intersections des courbes [14]. Dans SuperSVG, les auteurs combinent plusieurs pertes adaptées à la vectorisation par superpixels, notamment une perte de reconstruction normalisée, une perte de frontière pour empêcher les chemins de dépasser les limites du superpixel, et une perte Dynamic Path Warping permettant au modèle de raffinement d'hériter des connaissances du modèle grossier [12].

1.5.4 Apprentissage coarse-to-fine

L'apprentissage *coarse-to-fine* désigne une stratégie progressive dans laquelle le modèle commence par reconstruire les éléments globaux ou principaux d'une image, avant d'ajouter progressivement des détails plus fins. Dans le contexte de la vectorisation d'images, cette approche est particulièrement utile car une image complexe est difficile à convertir directement en une représentation SVG précise. En procédant par étapes, le modèle peut d'abord approximer la structure générale, puis améliorer localement la reconstruction. SuperSVG adopte cette logique à travers un cadre d'apprentissage en deux étapes : un modèle grossier prédit d'abord des chemins SVG pour reconstruire la structure principale du superpixel, puis un modèle de raffinement ajoute d'autres chemins afin d'enrichir les détails, en étant guidé par les chemins prédits à l'étape précédente [12]. Une idée similaire apparaît dans LIVE, où l'image est reconstruite progressivement en ajoutant et en optimisant de nouveaux chemins de Bézier dans une représentation en couches, ce qui permet d'obtenir un SVG plus compact et plus cohérent avec la structure visuelle de l'image [14].

1.6 Évaluation des résultats

L'évaluation d'une méthode de vectorisation consiste généralement à rendre le SVG généré sous forme matricielle, puis à comparer ce rendu à l'image d'entrée. Cette section présente les principales métriques utilisées dans nos expérimentations : la MSE, le PSNR, la SSIM, la LPIPS et le temps d'inférence.

1.6.1 MSE

La MSE (*Mean Squared Error*) mesure l'erreur moyenne entre les pixels de l'image originale I et ceux de l'image reconstruite $\hat{I}$:

$$\text{MSE}(I, \hat{I}) = \frac{1}{HWC} \sum_{i=1}^H \sum_{j=1}^W \sum_{c=1}^C (I_{i,j,c} - \hat{I}_{i,j,c})^2 \quad (1.2)$$

Dans l'équation (1.2), H et W représentent les dimensions de l'image et C son nombre de canaux. Une valeur faible indique une reconstruction plus fidèle au niveau des pixels. Cette métrique reste toutefois limitée, car elle ne correspond pas toujours à la qualité visuelle perçue [14, 12].

1.6.2 PSNR

Le PSNR (*Peak Signal-to-Noise Ratio*) est une métrique dérivée de la MSE. Pour des images dont les valeurs sont normalisées entre 0 et 1, il est défini par :

$$\text{PSNR}(I, \hat{I}) = 10 \log_{10} \left(\frac{1}{\text{MSE}(I, \hat{I})} \right) \quad (1.3)$$

Contrairement à la MSE, une valeur élevée du PSNR indique une meilleure fidélité de reconstruction. Cette métrique est principalement utilisée pour évaluer les différences pixel par pixel entre l'image cible et le rendu du SVG [11, 12].

1.6.3 SSIM

La SSIM (*Structural Similarity Index Measure*) mesure la similarité structurelle entre deux images en prenant en compte leur luminance, leur contraste et leur structure locale [15]. Elle est définie par :

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)} \quad (1.4)$$

Dans l'équation (1.4), μ_x et μ_y désignent les moyennes des deux fenêtres comparées, σ_x^2 et σ_y^2 leurs variances, et σ_{xy} leur covariance. Les constantes C_1 et C_2 assurent la stabilité du calcul. Une valeur élevée indique une meilleure préservation de la structure visuelle [12].

1.6.4 LPIPS

La LPIPS (*Learned Perceptual Image Patch Similarity*) compare deux images à partir de caractéristiques extraites par un réseau de neurones pré-entraîné, plutôt qu'à partir de leurs pixels uniquement [16]. Elle peut être exprimée sous la forme :

$$\text{LPIPS}(x, \hat{x}) = \sum_l \frac{1}{H_l W_l} \sum_{h,w} \|w_l \odot (\phi_l(x)_{h,w} - \phi_l(\hat{x})_{h,w})\|_2^2 \quad (1.5)$$

Dans l'équation (1.5), ϕ_l représente les caractéristiques extraites à la couche l du réseau, tandis que w_l désigne les poids associés à ces caractéristiques. Une faible valeur correspond à une meilleure similarité perceptuelle entre l'image d'entrée et le rendu SVG [12].

1.6.5 Temps d'inférence

Le temps d'inférence correspond à la durée nécessaire pour produire un SVG à partir d'une image d'entrée. Il permet d'évaluer l'efficacité pratique d'une méthode, indépendamment de sa qualité de reconstruction. Les méthodes reposant sur l'optimisation directe des paramètres vectoriels nécessitent généralement plusieurs itérations, tandis que les modèles entraînés peuvent prédire ces paramètres plus rapidement. Cette métrique est donc utilisée conjointement avec les mesures de qualité afin de comparer la précision et la rapidité des différentes approches [12].

1.7 Travaux connexes

La vectorisation automatique d'images a connu une évolution progressive, allant des approches algorithmiques classiques jusqu'aux travaux récents combinant apprentissage profond et rendu différentiable. Pour comparer ces travaux, il ne suffit pas de considérer uniquement la similarité visuelle entre l'image d'entrée et le SVG obtenu. Une approche de vectorisation doit également être évaluée selon plusieurs critères, notamment la fidélité visuelle, le temps de calcul, le nombre de chemins générés, la simplicité du SVG, son éditabilité, sa capacité de généralisation et le niveau de contrôle laissé à l'utilisateur [17]. Dans cette section, nous présentons donc les approches les plus pertinentes pour notre sujet, en montrant progressivement comment les limites des approches classiques et fondées sur l'optimisation ont conduit à des travaux plus récents comme SuperSVG.

1.7.1 Approches algorithmiques

Les approches algorithmiques reposent généralement sur l'analyse géométrique de l'image, comme l'extraction de contours, la détection de régions ou l'ajustement de courbes. Elles sont souvent rapides et ne nécessitent pas de données d'entraînement, mais restent limitées lorsque l'image contient des couleurs complexes, des textures ou un grand nombre de détails.

A. Potrace

Potrace est une méthode classique de vectorisation principalement conçue pour convertir des images binaires en contours vectoriels [18]. Son principe consiste à extraire les frontières entre les zones noires et blanches, à les approximer par des polygones, puis à transformer ces polygones en courbes de Bézier lisses. Cette méthode est simple, rapide et efficace pour des images comme les logos, les textes ou les dessins en noir et blanc. Cependant, elle dépend fortement de la qualité de la binarisation et reste peu adaptée aux images complexes, colorées ou texturées. Potrace constitue donc une

référence importante pour les approches classiques, mais montre aussi les limites des méthodes purement algorithmiques face à une vectorisation plus générale.

1.7.2 Approches basées sur l'optimisation

Les approches basées sur l'optimisation cherchent à ajuster directement les paramètres des primitives vectorielles afin que le rendu obtenu soit le plus proche possible de l'image d'entrée. Avec le rendu différentiable, il devient possible de calculer des gradients entre l'image rendue et l'image cible, puis de les utiliser pour modifier les courbes, les couleurs ou l'opacité des chemins SVG.

A. DiffVG

DiffVG propose un rasteriseur différentiable permettant de relier le domaine vectoriel au domaine matriciel [11]. Grâce à ce mécanisme, un ensemble de courbes ou de formes vectorielles peut être rendu sous forme d'image, comparé à une image cible, puis optimisé par rétropropagation. Cette approche est importante car elle permet d'utiliser une supervision uniquement matricielle, sans avoir besoin d'un SVG de référence. Elle est donc devenue une base technique pour plusieurs méthodes récentes. Cependant, DiffVG reste coûteux lorsqu'il faut optimiser beaucoup de chemins ou effectuer un grand nombre d'itérations. De plus, le nombre de primitives doit souvent être fixé à l'avance, ce qui limite le contrôle automatique de la complexité du SVG.

B. LIVE

LIVE, pour *Layer-wise Image Vectorization*, améliore les approches d'optimisation en introduisant une reconstruction progressive par couches [14]. La méthode ajoute progressivement des chemins de Bézier, puis les optimise afin de reconstruire l'image de manière *coarse-to-fine*. Cette stratégie permet d'obtenir des SVG plus structurés, plus compacts et plus éditables que ceux produits par une optimisation naïve. LIVE introduit également des pertes spécifiques pour mieux préserver les contours et limiter les auto-intersections des chemins. Malgré ces avantages, la méthode reste lente, car chaque image nécessite une optimisation itérative. Le résultat dépend aussi du nombre de chemins choisi et de plusieurs hyperparamètres. LIVE montre donc l'intérêt d'une représentation vectorielle progressive et éditable, mais confirme aussi le besoin de méthodes plus rapides.

1.7.3 Approches basées sur l'apprentissage profond

Les approches basées sur l'apprentissage profond cherchent à apprendre directement la relation entre une image matricielle et une représentation vectorielle. Contrairement

aux méthodes d'optimisation qui ajustent les chemins pour chaque image, ces méthodes entraînent un modèle capable de prédire les paramètres vectoriels à partir de l'image d'entrée, ce qui permet généralement une inférence plus rapide.

A. Im2Vec

Im2Vec propose une méthode de génération de graphiques vectoriels sans supervision vectorielle directe [19]. Contrairement aux approches qui nécessitent des fichiers SVG annotés, le modèle apprend uniquement à partir d'images matricielles. Son architecture suit un schéma encodeur-décodeur : l'image d'entrée est d'abord encodée dans un espace latent, puis ce code latent est utilisé pour générer une représentation vectorielle composée de plusieurs chemins fermés. Pour gérer les formes constituées de plusieurs composants, Im2Vec utilise un module récurrent de type RNN/LSTM qui produit un code latent propre à chaque chemin. Chaque chemin est ensuite décodé sous forme de courbes de Bézier à l'aide d'un décodeur fondé sur des convolutions 1D circulaires, ce qui permet de modéliser des formes fermées comme des déformations d'un cercle.

Le résultat vectoriel généré est ensuite rendu sous forme matricielle à l'aide d'un rendu différentiable, puis comparé à l'image cible afin d'entraîner le modèle sans supervision SVG explicite. Cette approche est intéressante car elle évite le besoin de jeux de données vectoriels annotés, qui sont difficiles à obtenir et souvent ambigus, puisqu'une même image peut être représentée par plusieurs structures vectorielles différentes. Cependant, Im2Vec reste surtout adapté à des images relativement simples ou appartenant à des domaines limités, comme les icônes, les caractères ou les emojis. Il montre donc l'intérêt de l'apprentissage profond pour la génération et la vectorisation de formes paramétriques, mais reste limité pour traiter des images complexes et variées.

B. SuperSVG

SuperSVG est la méthode la plus proche de notre thématique et constitue une référence importante dans les travaux récents de vectorisation image-to-SVG [12]. Son principe consiste d'abord à décomposer l'image d'entrée en superpixels, afin de réduire la difficulté du problème en travaillant sur des régions plus homogènes en couleur et en texture. Chaque superpixel est ensuite vectorisé séparément par un modèle d'apprentissage profond chargé de prédire les paramètres des chemins SVG correspondants.

L'architecture de SuperSVG repose sur un modèle en deux étapes. Dans la première étape, dite coarse-stage, un encodeur Vision Transformer extrait les caractéristiques visuelles du superpixel. Ces caractéristiques sont ensuite combinées avec des requêtes apprenables associées aux chemins SVG à l'aide d'un module de cross-attention. Un module de self-attention traite ensuite les représentations intermédiaires afin de les projeter

vers l'espace des paramètres vectoriels. Cette première étape permet de reconstruire la structure principale du superpixel. Dans la seconde étape, dite refinement-stage, un second modèle prend en entrée le rendu obtenu à partir des chemins grossiers ainsi que le superpixel cible, puis prédit des chemins supplémentaires destinés à enrichir les détails. Les sorties des deux étapes sont finalement combinées pour produire le SVG final.

SuperSVG combine ainsi plusieurs idées importantes des travaux précédents : le rendu différentiable hérité de DiffVG, la reconstruction progressive proche de LIVE, et la prédiction directe de chemins SVG par apprentissage profond. Cette architecture lui permet d'obtenir un meilleur compromis entre qualité de reconstruction et temps de calcul, notamment par rapport aux méthodes purement basées sur l'optimisation. Néanmoins, ses résultats dépendent encore de la qualité de la décomposition en superpixels, du nombre de chemins utilisés et de la capacité du modèle à représenter correctement les détails fins de chaque région. Malgré ces limites, SuperSVG représente l'approche la plus pertinente pour notre état de l'art, car elle illustre l'évolution récente des méthodes de vectorisation vers des solutions plus rapides, plus précises et mieux adaptées aux images complexes.

1.8 Synthèse comparative des approches

Les travaux étudiés montrent que la vectorisation automatique d'images repose sur un compromis entre plusieurs critères : la fidélité visuelle, le temps de calcul, la simplicité du SVG généré, le nombre de chemins, l'éditabilité et la capacité de généralisation. Les méthodes algorithmiques privilégient généralement la rapidité et la simplicité, mais restent limitées aux images peu complexes. Les méthodes basées sur l'optimisation offrent une meilleure flexibilité grâce au rendu différentiable, mais leur coût augmente avec le nombre de chemins et d'itérations. Les approches basées sur l'apprentissage profond cherchent à dépasser cette limite en apprenant à prédire directement les paramètres vectoriels.

TABLE 1.1 – Synthèse comparative des principales approches de vectorisation étudiées.

Approche	Principe	Points forts	Limites	Apport
Potrace	Tracé de contours binaires et approximation par courbes de Bézier.	Rapide, simple et sans apprentissage.	Limité aux images complexes, colorées ou texturées; dépend de la binarisation.	Base des approches algorithmiques classiques.

Approche	Principe	Points forts	Limites	Apport
DiffVG	Optimisation de primitives SVG via un rendu différentiable.	Permet une supervision matricielle et la rétropropagation vers les paramètres SVG.	Coûteux, sensible à l'initialisation et au nombre de chemins fixé.	Introduit le rendu différentiable comme outil central.
LIVE	Ajout progressif de chemins de Bézier selon une logique couche par couche.	Produit des SVG plus structurés, compacts et éditables.	Temps de calcul élevé ; dépend des chemins et des hyperparamètres.	Montre l'intérêt d'une reconstruction progressive.
Im2Vec	Prédiction de formes vectorielles à partir d'images matricielles.	Évite la supervision SVG directe et apprend un espace latent.	Surtout adapté aux images simples comme les icônes, caractères ou emojis.	Introduit l'apprentissage profond pour prédire des primitives vectorielles.
SuperSVG	Vectorisation par superpixels avec reconstruction <i>coarse-to-fine</i> .	Bon compromis entre qualité, vitesse et traitement d'images complexes.	Dépend de la décomposition en superpixels et du nombre de chemins.	Approche récente la plus proche de notre problématique.

Cette comparaison montre que SuperSVG constitue l'approche la plus complète parmi les travaux étudiés. Elle reprend le rendu différentiable introduit par DiffVG, la logique progressive mise en avant par LIVE, et l'efficacité des modèles d'apprentissage profond. Cependant, la vectorisation automatique reste confrontée à plusieurs difficultés, notamment le contrôle de la complexité du SVG, la fidélité visuelle, le temps de calcul et la généralisation à des images variées.

1.9 Conclusion

Dans ce chapitre, nous avons présenté les notions essentielles liées à la vectorisation d'images, puis étudié les principales familles de méthodes existantes. L'analyse des travaux connexes montre une évolution progressive des approches algorithmiques vers les méthodes d'optimisation différentiable, puis vers les modèles basés sur l'apprentissage profond.

La suite du mémoire présentera l'approche proposée pour répondre à cette problématique, ainsi que les choix de conception retenus pour la mise en place du système de vectorisation.

Chapitre 2

Approche proposée

2.1 Introduction

Le chapitre précédent a présenté les notions nécessaires à la compréhension de la vectorisation d’images ainsi que les principales familles de méthodes existantes. Nous décrivons à présent la solution mise en œuvre dans ce projet. Son objectif est de convertir une image matricielle couleur en un ensemble de chemins vectoriels fermés, représentés par des courbes de Bézier cubiques et enregistrés dans un fichier SVG. La méthode ne nécessite pas de fichiers SVG de référence pendant l’entraînement : la supervision est obtenue en rendant les chemins prédits avec DiffVG, puis en comparant le rendu à la région matricielle d’origine.

Notre approche repose sur une décomposition locale de l’image. L’image complète est d’abord segmentée avec SLIC, puis chaque superpixel est recadré et vectorisé séparément. Un encodeur visuel DINOv3 extrait les caractéristiques de la région, tandis qu’un décodeur fondé sur la cross-attention et la self-attention prédit directement les points de contrôle et la couleur de chaque chemin. Cette organisation reprend l’idée de simplifier une image complexe par superpixels, mise en avant dans SuperSVG, mais la version actuellement implémentée utilise une seule étape de prédiction et ne comporte pas de modèle de raffinement *coarse-to-fine* [12].

Dans ce chapitre, nous présentons d’abord la vue d’ensemble de la méthode et la préparation des données. Nous détaillons ensuite l’architecture du modèle, la représentation des primitives vectorielles, le rendu différentiable et les fonctions de perte. Enfin, nous décrivons la stratégie d’entraînement ainsi que le passage des prédictions locales au SVG de l’image complète.

2.2 Vue d’ensemble de la méthode

Cette section présente l’organisation générale de l’approche proposée et les principaux choix ayant guidé sa conception. Nous décrivons d’abord le cheminement suivi depuis la décomposition de l’image en superpixels jusqu’à la prédiction et à l’assemblage des primitives vectorielles. Nous précisons ensuite les choix retenus concernant la supervision matricielle, la prédiction des chemins en une seule passe et les principaux modules constituant la pipeline.

2.2.1 Principe général

La conversion directe d’une image naturelle complète vers un nombre limité de primitives vectorielles constitue un problème difficile. Une même image peut contenir de nombreux objets, couleurs, contours et détails, tandis qu’il existe plusieurs descriptions SVG capables de produire un rendu visuellement proche. Nous réduisons cette difficulté en travaillant sur des régions dont les pixels sont plus homogènes. Comme les superpixels tendent à suivre les changements de couleur et les frontières locales, chaque prédiction porte sur une forme plus simple que l’image entière. Cette décomposition a été introduite dans la section 1.5.1 ; elle est appliquée ici avec l’algorithme SLIC [13].

Pour chaque région, le réseau prédit un nombre fixé P de chemins. Chaque chemin est décrit par douze points bidimensionnels et une couleur RGB, soit un vecteur de 27 paramètres. Les chemins sont ensuite rasterisés par DiffVG. Pendant l’entraînement, cette image rendue permet de calculer une erreur de reconstruction et une pénalité lorsque les formes débordent du superpixel. Pendant l’inférence, les coordonnées prédites dans le repère local du recadrage sont replacées dans le repère de l’image originale, puis l’ensemble des chemins est enregistré au format SVG.

La figure 2.1 résume ce flux. Le trajet supérieur est commun à l’entraînement et à l’inférence. Les deux branches inférieures montrent, d’une part, l’utilisation du rendu différentiable pour apprendre les paramètres du réseau et, d’autre part, la transformation géométrique utilisée pour construire le document vectoriel final.

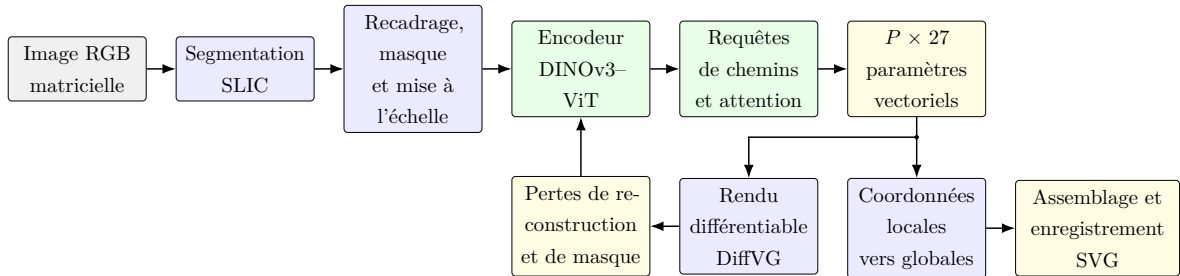


FIGURE 2.1 – Pipeline générale de l’approche proposée, de l’image matricielle aux chemins SVG.

2.2.2 Choix de conception

Le premier choix consiste à apprendre à partir d’images matricielles uniquement. La correspondance entre une image et un SVG n’est pas unique : deux ensembles de courbes différents peuvent produire des rendus proches. Exiger une vérité terrain vectorielle imposerait donc une structure particulière sans garantir qu’elle soit la seule représentation correcte. Le rendu différentiable permet d’éviter cette contrainte en ramenant la supervision dans l’espace des pixels, comme dans DiffVG et plusieurs méthodes étudiées au chapitre 1 [11, 19].

Le second choix est de prédire les primitives en une seule passe du réseau. Contrairement à une optimisation directe qui ajuste les courbes séparément pour chaque nouvelle image, les paramètres sont ici produits par un modèle entraîné. L'inférence ne lance donc pas de descente de gradient sur les chemins. Cette conception vise à séparer le coût de l'apprentissage, effectué une fois, du traitement des images ultérieures. La version implémentée reste volontairement simple : elle emploie un seul décodeur, un nombre fixe de chemins par région et deux termes de perte directement liés à la reconstruction et au respect du masque.

Le tableau 2.1 présente les principaux modules et les données qu'ils échangent. Les dimensions indiquées correspondent à une région redimensionnée en 224×224 pixels. La dimension interne d est déterminée automatiquement à partir du modèle DINOv3 chargé.

TABLE 2.1 – Principaux modules de la pipeline proposée.

Module	Entrée et sortie	Rôle dans la pipeline
SLIC	Image RGB $\rightarrow$ carte d'étiquettes	Décomposer l'image en régions localement homogènes et connexes.
Prétraitement	Région et masque $\rightarrow$ tenseur $3 \times 224 \times 224$	Recadrer le superpixel, normaliser sa taille et distinguer la région de son arrière-plan.
DINOv3-ViT	Région $\rightarrow 196 \times d$ tokens	Extraire une représentation visuelle dense des 14×14 patches.
Décodeur d'attention	Tokens visuels et P requêtes $\rightarrow P \times d$	Associer chaque requête de chemin aux zones pertinentes, puis modéliser les interactions entre chemins.
Tête linéaire	$P \times d \rightarrow P \times 27$	Prédire les coordonnées normalisées et la couleur de remplissage.
DiffVG	Chemins vectoriels $\rightarrow$ image RGBA	Fournir un rendu différentiable pour le calcul des pertes et un aperçu matriciel à l'inférence.
Export SVG	Chemins globaux $\rightarrow$ fichier SVG	Enregistrer les primitives à la résolution de l'image d'origine.

2.3 Préparation des données

Cette section décrit les opérations appliquées aux images avant leur passage dans le modèle. Nous présentons d'abord le jeu de données utilisé, puis la décomposition des images en superpixels et, enfin, la construction des recadrages locaux fournis au réseau

avec leur masque associé.

2.3.1 Jeu de données utilisé

L'implémentation utilise le sous-ensemble d'entraînement d'ImageNet comme source d'images naturelles [20]. Les fichiers sont parcourus récursivement et chargés en RGB, puis leurs valeurs entières sont converties en nombres réels dans l'intervalle $[0, 1]$. Le chargeur accepte les principaux formats matriciels, notamment JPEG, PNG, BMP, WebP et TIFF. Aucune étiquette de classe ImageNet n'intervient dans l'objectif : seules les images sont exploitées pour apprendre la reconstruction de régions visuelles.

Un exemple du jeu de données ne correspond pas à une image complète. À chaque accès, l'image concernée est segmentée, l'ordre de ses superpixels est mélangé et une région non vide est choisie. Cette stratégie permet de présenter différentes régions d'une même image au cours des époques sans stocker à l'avance tous les recadrages. En cas d'image corrompue ou de région impossible à traiter, le chargeur tente jusqu'à cinq images voisines afin qu'une erreur isolée n'interrompe pas un entraînement long.

2.3.2 Décomposition en superpixels

Pour une image $I \in [0, 1]^{H_0 \times W_0 \times 3}$, SLIC produit une carte d'étiquettes $L \in \mathbb{N}^{H_0 \times W_0}$. Les pixels portant la même étiquette définissent un masque binaire. Le code demande 16 régions pendant l'entraînement, avec une compacité de 10 et un lissage gaussien de paramètre $\sigma = 1$. La connectivité est imposée afin d'éviter qu'une même étiquette corresponde à plusieurs composantes éloignées. Le nombre demandé est également borné entre 2 et $H_0 W_0$ pour conserver une valeur valide sur les petites images.

Ce choix fournit un compromis entre la taille des régions et leur homogénéité. Une valeur faible du nombre de segments laisserait au réseau des régions complexes, alors qu'une valeur très élevée multiplierait le nombre de chemins nécessaires pour reconstruire une image entière. La compacité contrôle quant à elle l'équilibre entre la proximité spatiale et la similarité colorimétrique dans SLIC [13]. Les effets de ces paramètres sur la qualité et le coût de la vectorisation seront évalués séparément dans le chapitre consacré aux expérimentations.

2.3.3 Recadrage et construction de l'entrée

Pour un superpixel donné, nous calculons la plus petite boîte englobante (x_1, y_1, x_2, y_2) contenant tous ses pixels. L'image et le masque sont découpés selon cette boîte. Le recadrage RGB est ensuite redimensionné en 224×224 par interpolation bilinéaire, tandis que le masque utilise l'interpolation du plus proche voisin afin de rester binaire.

Cette différence est importante : une interpolation continue du masque créerait des valeurs intermédiaires qui rendraient ambiguë l'appartenance à la région.

Soient $Y \in [0, 1]^{3 \times H \times W}$ le recadrage RGB redimensionné et $M \in \{0, 1\}^{1 \times H \times W}$ son masque, avec $H = W = 224$. Le tenseur fourni au réseau est défini par :

$$X = Y \odot M - (1 - M), \quad (2.1)$$

où le masque est diffusé sur les trois canaux. Les pixels appartenant au superpixel conservent ainsi leur valeur RGB, alors que les pixels extérieurs prennent la valeur -1 . Le réseau reçoit donc à la fois l'apparence de la région et une indication explicite de sa forme. Le tenseur cible reste Y , mais la perte de reconstruction sera calculée uniquement là où $M = 1$. Le masque est conservé séparément pour normaliser cette perte et pénaliser les débordements.

Aucune augmentation par rotation, miroir ou modification colorimétrique n'est appliquée dans la version actuelle du chargeur. La variabilité provient du contenu d'ImageNet et du choix aléatoire du superpixel.

2.4 Architecture du modèle

Cette section décrit l'architecture chargée de transformer les caractéristiques visuelles d'un superpixel en paramètres vectoriels. Nous présentons d'abord l'extraction des représentations avec DINOv3, puis le décodage fondé sur des requêtes apprenables, la cross-attention et la self-attention. Enfin, nous détaillons la représentation adoptée pour chaque chemin vectoriel et la manière dont ses paramètres sont interprétés sous forme de courbes de Bézier cubiques.

2.4.1 Extraction des caractéristiques avec DINOv3

L'encodeur visuel utilisé est la variante `vit_small_patch16_dinov3`, chargée avec des poids pré-entraînés. DINOv3 est une famille de modèles visuels pré-entraînés par apprentissage auto-supervisé et conçus pour produire des représentations denses transférables [21]. Le recours à un modèle pré-entraîné est adapté à notre problème, car la reconstruction vectorielle nécessite de reconnaître des structures locales, des contours et des relations spatiales sans disposer d'annotations SVG.

Avant l'encodage, chaque canal de X est standardisé avec les moyennes $(0,485, 0,456, 0,406)$ et les écarts-types $(0,229, 0,224, 0,225)$ employés par l'implémentation. Si la taille de l'entrée diffère de 224×224 , une interpolation bilinéaire la ramène à cette résolution. Comme la taille d'un patch est de 16×16 , la région est divisée en une grille de 14×14 , soit 196 patches. Ce principe de représentation d'une image comme une séquence de patches a été présenté dans la section C. [10].

Le passage dans DINOv3 peut être résumé par :

$$F = E_\theta(X) \in \mathbb{R}^{196 \times d}, \quad (2.2)$$

où E_θ désigne l'encodeur visuel et d sa dimension d'embedding. La sortie de l'encodeur contient également des tokens préfixes, tels que le token de classification et, selon la variante chargée, des tokens registres. Le code retire tous ces préfixes et conserve uniquement les 196 tokens associés aux patches. Il vérifie explicitement leur nombre afin d'éviter de transmettre au décodeur une séquence incompatible avec la résolution prévue.

Les paramètres de DINOv3 ne sont pas gelés pendant l'entraînement. L'optimiseur reçoit l'ensemble des paramètres de l'encodeur de chemins, ce qui inclut le backbone pré-entraîné, les requêtes apprenables, les blocs d'attention et la tête de sortie. Le modèle bénéficie donc de l'initialisation auto-supervisée, puis adapte ses caractéristiques à la prédiction de primitives vectorielles.

2.4.2 Requêtes apprenables et cross-attention

Le décodeur maintient P vecteurs apprenables, appelés requêtes ou tokens de chemins. Ils forment une matrice initiale $S^{(0)} \in \mathbb{R}^{P \times d}$, identique pour toutes les images avant son interaction avec les caractéristiques visuelles. Chaque requête est destinée à produire un chemin, mais aucune association fixe n'est imposée entre une requête et une position de l'image. Cette association est apprise par la cross-attention.

Pour chaque tête d'attention, les requêtes proviennent des tokens de chemins, tandis que les clés et les valeurs proviennent des patches DINOv3. En reprenant la notation introduite dans la section 1.4, le calcul s'écrit :

$$Q = \text{LN}(S^{(0)}) W_Q, \quad (2.3)$$

$$K = \text{LN}(F) W_K, \quad V = \text{LN}(F) W_V, \quad (2.4)$$

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d_h}}\right) V, \quad (2.5)$$

où d_h est la dimension d'une tête. Les sorties des différentes têtes sont concaténées par le module d'attention multi-têtes. Une connexion résiduelle ajoute ensuite l'information obtenue aux requêtes initiales. Le résultat traverse un réseau *feed-forward* composé de deux projections linéaires, d'une activation GELU et d'une dimension cachée quatre fois supérieure à d . Une seconde connexion résiduelle produit les tokens contextualisés :

$$S^{(1)} = S^{(0)} + \text{MHA}_{\text{cross}}(S^{(0)}, F, F), \quad \tilde{S}^{(1)} = S^{(1)} + \text{FFN}(\text{LN}(S^{(1)})). \quad (2.6)$$

L'implémentation demande huit têtes. Si d n'est pas divisible par huit, leur nombre est automatiquement diminué jusqu'à obtenir un diviseur valide. Cette précaution permet de conserver le même code avec plusieurs variantes de backbone.

2.4.3 Interactions entre les chemins

Après la cross-attention, chaque requête contient principalement l'information visuelle utile à son propre chemin. Une couche de self-attention est ensuite appliquée entre les P requêtes. Les requêtes, les clés et les valeurs proviennent alors toutes de $\tilde{S}^{(1)}$. Cette opération permet à un chemin de tenir compte des autres chemins déjà représentés, ce qui peut limiter la redondance et répartir la reconstruction entre plusieurs primitives.

Le bloc suit la même organisation pré-normalisée que le bloc précédent : LayerNorm, attention multi-têtes, connexion résiduelle, puis réseau *feed-forward* et seconde connexion résiduelle. La version actuelle emploie exactement un bloc de cross-attention et un bloc de self-attention. Elle ne contient ni décodeur récurrent, ni empilement de plusieurs étapes de raffinement.

Une normalisation finale et une couche linéaire transforment chaque token en 27 valeurs. Une fonction sigmoïde borne toutes les sorties entre 0 et 1 :

$$Z = \sigma(W_o \text{LN}(S^{(2)}) + b_o) \in [0, 1]^{P \times 27}. \quad (2.7)$$

Cette contrainte convient simultanément aux coordonnées normalisées et aux trois composantes de couleur. Elle évite qu'un point soit placé hors du canevas local avant l'application de la perte de masque.

2.4.4 Représentation d'un chemin vectoriel

Pour le chemin p , le vecteur prédit est décomposé de la manière suivante :

$$z_p = (x_0, y_0, \dots, x_{11}, y_{11}, r_p, g_p, b_p). \quad (2.8)$$

Les 24 premières valeurs représentent douze points $P_0, \dots, P_{11}$ dans le repère normalisé du recadrage. Les trois dernières valeurs définissent la couleur de remplissage. DiffVG interprète ces points comme quatre segments de Bézier cubiques reliés bout à bout et ferme le dernier segment sur le point initial. Les points P_0 , P_3 , P_6 et P_9 sont les points d'ancrage, tandis que les autres points contrôlent la tangente et la courbure, comme l'illustre la figure 2.2.

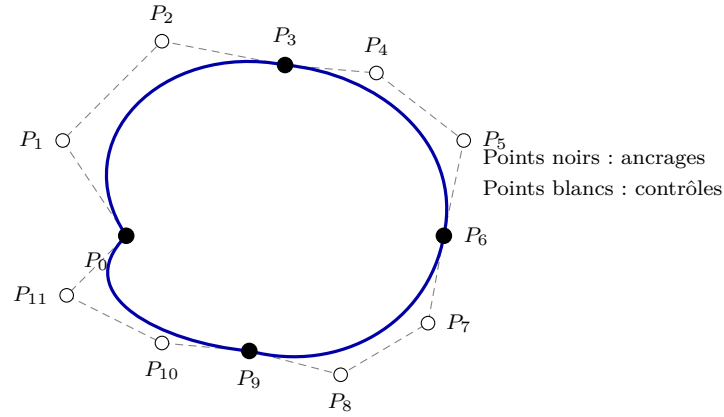


FIGURE 2.2 – Chemin fermé composé de quatre courbes de Bézier cubiques et de douze points prédits.

Pour quatre points consécutifs A , C_1 , C_2 et B , chaque segment cubique est défini, pour $t \in [0, 1]$, par :

$$\mathcal{B}(t) = (1 - t)^3 A + 3(1 - t)^2 t C_1 + 3(1 - t) t^2 C_2 + t^3 B. \quad (2.9)$$

Cette équation complète le rappel sur les courbes de Bézier de la section A. en précisant la structure réellement produite par le modèle. Dans le premier segment, par exemple, $A = P_0$, $C_1 = P_1$, $C_2 = P_2$ et $B = P_3$. Le quatrième segment part de P_9 , utilise P_{10} et P_{11} comme points de contrôle, puis revient à P_0 pour fermer la forme.

Les primitives sont remplies avec la couleur (r_p, g_p, b_p) . Leur opacité est fixée à 1 et leur largeur de contour à 0 ; le modèle produit donc des surfaces colorées sans trait séparé. Le nombre de chemins P est un hyperparamètre fixé avant la construction du réseau. La valeur par défaut du code est 32, tandis qu’une configuration d’entraînement enregistrée dans le projet utilise 128 chemins par superpixel. Aucune sortie ne prédit actuellement la visibilité, l’opacité ou un symbole d’arrêt permettant de faire varier ce nombre.

2.5 Rendu différentiable et fonctions de perte

Cette section présente le mécanisme utilisé pour comparer les primitives vectorielles prédites à la région matricielle cible. Nous décrivons d’abord la construction de la scène et sa rasterisation avec DiffVG, puis la perte de reconstruction calculée à l’intérieur du superpixel et la pénalité appliquée aux formes qui dépassent son masque. Enfin, nous définissons l’objectif global utilisé pour entraîner l’ensemble du réseau.

2.5.1 Construction de la scène DiffVG

Les paramètres Z sont convertis en objets `Path` et `ShapeGroup` de DiffVG. Pour un canevas local de largeur W et de hauteur H , chaque coordonnée normalisée (x, y) est multipliée par (W, H) . Les quatre segments reçoivent chacun deux points de contrôle, le chemin est déclaré fermé et sa couleur de remplissage est obtenue à partir des trois dernières valeurs du vecteur. Le rasteriseur différentiable calcule ensuite une image RGBA avec un échantillonnage 2×2 par pixel [11].

Si $\mathcal{R}$ désigne ce rendu, l'image RGB employée pour la reconstruction est composée sur un fond noir :

$$\hat{Y} = \mathcal{R}_{\text{RGB}}(Z) \odot \mathcal{R}_{\alpha}(Z). \quad (2.10)$$

La différentiabilité de $\mathcal{R}$ permet de transmettre le gradient de l'erreur pixel jusqu'aux coordonnées et aux couleurs, puis à l'ensemble du réseau. DiffVG est exécuté en précision simple, même lorsque la précision mixte est active autour du reste du modèle, afin de conserver un calcul de rendu compatible et stable.

2.5.2 Perte de reconstruction normalisée

La première composante de l'objectif mesure l'écart quadratique entre le rendu et la cible à l'intérieur du superpixel. Comme les régions n'occupent pas toutes la même proportion de leur boîte englobante, la somme est normalisée par la surface du masque. Pour un lot de taille B , nous définissons :

$$a_i = \frac{1}{HW} \sum_{u=1}^H \sum_{v=1}^W M_{i,u,v}, \quad (2.11)$$

puis :

$$\mathcal{L}_{\text{rec}} = \frac{1}{B} \sum_{i=1}^B \frac{1}{3HW \max(a_i, \frac{1}{HW})} \sum_{c=1}^3 \sum_{u=1}^H \sum_{v=1}^W M_{i,c,u,v} \left(\hat{Y}_{i,c,u,v} - Y_{i,c,u,v} \right)^2. \quad (2.12)$$

Cette formulation correspond à une MSE calculée sur les pixels utiles, indépendamment de la taille relative de la région. Sans la division par a_i , un superpixel occupant une faible partie du recadrage contribuerait moins à l'objectif qu'une région remplissant presque toute sa boîte. La borne inférieure $1/(HW)$ protège le calcul contre une division par zéro, même si le prétraitement vérifie déjà que le masque n'est pas vide.

2.5.3 Pénalité de débordement

La perte de reconstruction ne contraint pas les pixels extérieurs au masque, puisqu'ils sont multipliés par zéro. Un chemin pourrait donc dépasser largement la région tout en reconstruisant correctement sa partie visible à l'intérieur. Pour limiter ce comportement, les mêmes formes sont rendues une seconde fois en blanc et leur canal alpha est comparé à l'extérieur du superpixel. Cette idée est proche de la perte de frontière proposée dans SuperSVG [12].

Avant le calcul, le masque est dilaté par une opération de maximum local de rayon deux pixels. Soit $\widetilde{M}_i = \mathcal{D}_2(M_i)$ ce masque dilaté et $O_i = 1 - \widetilde{M}_i$ la zone extérieure. Si $\widehat{A}_i$ est le canal alpha du rendu blanc, la pénalité non pondérée vaut :

$$\mathcal{L}_{\text{ext}} = \frac{1}{B} \sum_{i=1}^B \frac{\sum_{u,v} \widehat{A}_{i,u,v} O_{i,u,v}}{\max\left(1, \sum_{u,v} O_{i,u,v}\right)}. \quad (2.13)$$

La dilatation crée une tolérance autour de la frontière. Elle évite de pénaliser trop fortement les pixels d'anticrénelage ou un léger décalage entre le masque redimensionné et le bord de la courbe. La normalisation par la surface extérieure rend la valeur comparable entre des masques de formes différentes.

2.5.4 Objectif global

La fonction objectif combine les deux termes précédents :

$$\mathcal{L} = \mathcal{L}_{\text{rec}} + \lambda_{\text{masque}} \mathcal{L}_{\text{ext}}. \quad (2.14)$$

Le poids de la pénalité décroît linéairement au fil des époques. Pour une époque $e \in \{0, \dots, E-1\}$ et $E > 1$, il est défini par :

$$\lambda_{\text{masque}}(e) = 0,05 + (0,01 - 0,05) \frac{e}{E-1}. \quad (2.15)$$

La contrainte spatiale est ainsi plus forte au début, lorsque les formes prédites sont encore peu structurées, puis son poids diminue pour laisser davantage de place à l'ajustement des couleurs et des contours. L'objectif ne contient actuellement ni perte perceptuelle LPIPS, ni SSIM, ni terme de contrôle du nombre de chemins. Ces mesures sont utiles pour l'évaluation décrite au chapitre précédent, mais elles ne participent pas à la rétropropagation de la version entraînée.

2.6 Stratégie d'entraînement

Cette section décrit le protocole utilisé pour entraîner le modèle et assurer la continuité des expériences. Nous présentons d'abord les choix d'optimisation, l'ordonnancement du taux d'apprentissage et les paramètres techniques retenus, puis le mécanisme de sauvegarde permettant de reprendre un entraînement interrompu et de restaurer automatiquement sa configuration.

2.6.1 Optimisation des paramètres

L'entraînement utilise AdamW avec un taux d'apprentissage initial de 10^{-4} et une décroissance de poids de 0,05. Le taux d'apprentissage augmente d'abord linéairement pendant une phase de *warm-up*, puis suit une décroissance cosinus jusqu'à la fin de l'entraînement. Tous les paramètres de l'encodeur de chemins sont optimisés conjointement. La graine aléatoire est appliquée à Python, NumPy, PyTorch et CUDA afin de rendre les tirages aussi reproductibles que possible.

La précision mixte automatique est activée lorsque l'entraînement s'exécute sur GPU. Un facteur d'échelle dynamique est appliqué à la perte avant la rétropropagation, puis l'optimiseur et l'ordonnanceur sont mis à jour à chaque lot. Le code permet une limitation optionnelle de la norme du gradient, mais aucune valeur n'est définie dans les configurations enregistrées ; cette limitation est donc inactive pour les entraînements concernés.

Le tableau 2.2 reprend la configuration complète enregistrée pour l'entraînement à 128 chemins. Il décrit le protocole technique, sans anticiper l'analyse des performances qui appartient au chapitre suivant.

TABLE 2.2 – Configuration enregistrée pour l’entraînement du modèle à 128 chemins par superpixel.

Paramètre	Valeur
Source des images	ImageNet, avec un maximum de 10 000 images
Taille d’une région	224×224 pixels
Paramètres SLIC à l’entraînement	16 régions, compacité 10, $\sigma = 1$
Backbone	vit_small_patch16_dinov3, poids pré-entraînés
Nombre de chemins par région	128
Taille des lots	16 régions
Nombre d’époques	20
Optimiseur	AdamW, taux 10^{-4} , décroissance de poids 0,05
Ordonnancement	<i>Warm-up</i> de 4 époques, puis décroissance cosinus
Précision mixte	Activée sur CUDA
Graine aléatoire	1234
Poids de la perte de masque	Décroissance linéaire de 0,05 à 0,01

2.6.2 Sauvegarde et reprise

À la fin de chaque époque, le programme enregistre l’état du modèle, de l’optimiseur, de l’ordonnanceur et du facteur d’échelle de la précision mixte. Les arguments de l’expérience sont également conservés dans le point de contrôle et dans un fichier JSON séparé. Deux formes de sauvegarde sont produites : un fichier numéroté pour chaque époque et un fichier `latest.pt` représentant le dernier état écrit.

Cette organisation permet de reprendre un entraînement sans réinitialiser le taux d’apprentissage ni les statistiques de l’optimiseur. Lors d’une reprise, l’époque suivante est calculée à partir du numéro sauvegardé et tous les états sont restaurés. Elle permet également au programme d’inférence de reconstruire automatiquement le nombre de chemins, la taille du canevas et le nom du backbone associés au modèle chargé.

2.7 Inférence et génération du SVG

Cette section décrit le passage du modèle entraîné à la génération du fichier SVG final. Nous présentons d’abord le traitement de l’image complète et la transformation des coordonnées locales en coordonnées globales, puis l’algorithme d’inférence utilisé

pour assembler les chemins prédits. Nous comparons ensuite les flux d’entraînement et d’inférence avant de préciser les limites de la version actuellement implémentée.

2.7.1 Traitement de l’image complète

Pendant l’entraînement, une seule région aléatoire est extraite de chaque image chargée. À l’inférence, l’objectif est au contraire de couvrir l’image entière. SLIC est donc appliqué une fois, puis toutes les étiquettes non vides sont parcourues. La configuration par défaut demande 64 régions, avec la même compacité 10 et le même $\sigma = 1$. Chaque région est préparée de la même manière qu’à l’entraînement et les recadrages sont regroupés par lots de 16 avant leur passage dans le réseau.

Le modèle retourne des coordonnées normalisées dans la boîte locale de chaque superpixel. Pour un point prédit $(u, v) \in [0, 1]^2$ et une boîte (x_1, y_1, x_2, y_2) , le passage au repère de l’image originale est effectué par :

$$x = x_1 + (x_2 - x_1)u, \quad y = y_1 + (y_2 - y_1)v. \quad (2.16)$$

Cette transformation est appliquée aux douze points de chaque chemin, tandis que la couleur reste inchangée. Les chemins de toutes les régions sont ensuite concaténés. Si SLIC produit effectivement R régions et que le modèle utilise P requêtes, le SVG contient donc $R \times P$ chemins. La version actuelle ne supprime aucun chemin après la prédiction et ne fusionne pas les formes similaires.

2.7.2 Algorithme d’inférence

L’algorithme 1 résume les opérations réalisées. Le rendu PNG est produit à la résolution originale pour permettre une inspection rapide. Le fichier SVG est enregistré à partir des mêmes chemins globaux, sans conversion intermédiaire en pixels.

Algorithm 1 Vectorisation d'une image complète et génération du SVG.

Require: Image RGB I , modèle entraîné f_θ , nombre de régions R
Ensure: Fichier SVG et aperçu PNG

```

1:  $L \leftarrow \text{SLIC}(I, R)$ 
2:  $S_{\text{global}} \leftarrow \emptyset$ 
3: for all étiquettes  $k$  présentes dans  $L$  do
4:    $M_k \leftarrow [L = k]$ 
5:    $(X_k, b_k) \leftarrow \text{PRÉPARERRÉGION}(I, M_k)$ 
6:    $S_k \leftarrow f_\theta(X_k)$ 
7:    $S_k^g \leftarrow \text{LOCALVERSGLOBAL}(S_k, b_k)$ 
8:    $S_{\text{global}} \leftarrow S_{\text{global}} \cup S_k^g$ 
9: end for
10:  $\hat{I} \leftarrow \text{DIFFVG}(S_{\text{global}}, \text{taille}(I))$ 
11:  $\text{ENREGISTRERPNG}(\hat{I})$ 
12:  $\text{ENREGISTRERSVG}(S_{\text{global}}, \text{taille}(I))$ 

```

En pratique, les régions sont traitées par lots plutôt qu'une par une comme dans ce pseudo-code, mais cette optimisation ne change pas le résultat. Le calcul est exécuté sans gradient et peut utiliser la précision mixte sur GPU. Lorsqu'un point de contrôle est chargé, les paramètres architecturaux qu'il contient sont prioritaires, sauf si l'utilisateur fournit explicitement une valeur différente sur la ligne de commande.

2.7.3 Différences entre entraînement et inférence

Le tableau 2.3 met en évidence les différences qui découlent du rôle de chaque phase. Le même prétraitement local et le même réseau sont conservés, mais la sélection des régions et l'utilisation des sorties changent.

TABLE 2.3 – Comparaison du flux d’entraînement et du flux d’inférence.

Élément	Entraînement	Inférence
Segmentation par défaut	16 superpixels demandés par image	64 superpixels demandés par image
Régions traitées	Une région aléatoire par image et par accès	Toutes les régions non vides de l’image
Repère des points	Coordonnées locales normalisées	Coordonnées replacées dans l’image originale
DiffVG	Produit le rendu utilisé par les pertes	Produit uniquement l’aperçu PNG
Sortie vectorielle	Pas de fichier SVG dans la boucle d’apprentissage	Tous les chemins sont assemblés et enregistrés
Gradient	Activé jusqu’au backbone DINOv3	Désactivé

2.7.4 Périmètre de la version implémentée

La méthode mise en œuvre couvre la chaîne complète, depuis une image RGB jusqu’à un fichier SVG éditible, mais certains choix délimitent son comportement. Le nombre de chemins est fixé pour toutes les régions, indépendamment de leur complexité. Une zone presque uniforme reçoit donc autant de primitives qu’une zone contenant plusieurs variations de couleur. De plus, les chemins sont toujours fermés, opaques et remplis ; les traits ouverts, les contours d’épaisseur variable, les dégradés et la transparence ne sont pas représentés.

La séparation en superpixels rend le problème local plus accessible, mais elle rend également le résultat dépendant de SLIC. Une frontière mal placée peut répartir un même objet entre plusieurs recadrages. La perte de masque encourage les formes à rester dans leur région, sans toutefois appliquer un découpage géométrique strict au moment de l’export. Enfin, la version actuelle ne contient pas l’étape de raffinement, la perte de transfert entre étapes ou l’optimisation finale des courbes présentes dans la méthode SuperSVG complète [12].

Ces éléments ne constituent pas des résultats expérimentaux ; ils décrivent le périmètre exact du système évalué. Leur influence sur la fidélité du rendu, le nombre de chemins et le temps d’inférence devra être mesurée dans le chapitre suivant à l’aide des métriques présentées au chapitre 1.

2.8 Conclusion

Dans ce chapitre, nous avons présenté l’approche développée pour la vectorisation automatique d’images matricielles. La méthode décompose d’abord l’image avec SLIC, prépare chaque superpixel dans un repère local, puis utilise un encodeur DINOv3 et un décodeur d’attention pour prédire des chemins fermés composés de courbes de Bézier cubiques. Chaque primitive est décrite par douze points de contrôle et une couleur de remplissage.

L’apprentissage ne nécessite pas de SVG cible. DiffVG transforme les prédictions en images et permet d’optimiser une perte de reconstruction normalisée ainsi qu’une pénalité de débordement hors du masque. À l’inférence, les chemins locaux sont replacés dans le repère de l’image originale, assemblés et enregistrés directement au format SVG. Le chapitre suivant sera consacré au protocole expérimental et à l’évaluation quantitative et qualitative de cette architecture.

Chapitre 3

Expérimentations et Résultats

Conclusion Générale

Au cours de ce projet de fin d'étude, nous avons exploré divers aspects de l'apprentissage profond et de la vision par ordinateur pour résoudre le problème du redimensionnement des images.

Bibliographie

- [1] Rafael C. GONZALEZ et Richard E. WOODS. *Digital Image Processing*. 4th. Pearson, 2018. ISBN : 978-1-292-22304-9.
- [2] James D. FOLEY et al. *Computer Graphics : Principles and Practice*. 2nd. Addison-Wesley Professional, 1996, p. 8.
- [3] W3C. *Scalable Vector Graphics (SVG) 1.1 (Second Edition)*. W3C Recommendation. World Wide Web Consortium, 2011. URL : <https://www.w3.org/TR/SVG11/>.
- [4] Aurélien GÉRON. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3rd. O'Reilly Media, 2022. ISBN : 978-1-098-12597-4.
- [5] Tom M. MITCHELL. *Machine Learning*. McGraw-Hill, 1997, p. 2.
- [6] Christopher M. BISHOP. *Pattern Recognition and Machine Learning*. Springer, 2006, p. 3.
- [7] Longlong JING et Yingli TIAN. “Self-Supervised Visual Feature Learning with Deep Neural Networks : A Survey”. In : *IEEE Transactions on Pattern Analysis and Machine Intelligence* 43.11 (2021), p. 4037-4058. DOI : 10.1109/TPAMI.2020.2992393.
- [8] Yann LECUN et al. “Backpropagation Applied to Handwritten Zip Code Recognition”. In : *Neural Computation* 1.4 (1989), p. 541-551. DOI : 10.1162/neco.1989.1.4.541.
- [9] Ashish VASWANI et al. “Attention Is All You Need”. In : *Advances in Neural Information Processing Systems*. T. 30. Curran Associates, Inc., 2017.
- [10] Alexey DOSOVITSKIY et al. “An Image Is Worth 16x16 Words : Transformers for Image Recognition at Scale”. In : *International Conference on Learning Representations*. 2021. URL : <https://arxiv.org/abs/2010.11929>.
- [11] Tzu-Mao LI et al. “Differentiable Vector Graphics Rasterization for Editing and Learning”. In : *ACM Transactions on Graphics* 39.6 (2020), 193 :1-193 :15. DOI : 10.1145/3414685.3417871.
- [12] Teng HU et al. “SuperSVG : Superpixel-based Scalable Vector Graphics Synthesis”. In : *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2024, p. 24892-24901.

- [13] Radhakrishna ACHANTA et al. “SLIC Superpixels Compared to State-of-the-Art Superpixel Methods”. In : *IEEE Transactions on Pattern Analysis and Machine Intelligence* 34.11 (2012), p. 2274-2282. DOI : 10.1109/TPAMI.2012.120.
- [14] Xu MA et al. “Towards Layer-wise Image Vectorization”. In : *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2022, p. 16314-16323. DOI : 10.1109/CVPR52688.2022.01583.
- [15] Zhou WANG et al. “Image Quality Assessment : From Error Visibility to Structural Similarity”. In : *IEEE Transactions on Image Processing* 13.4 (2004), p. 600-612. DOI : 10.1109/TIP.2003.819861.
- [16] Richard ZHANG et al. “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric”. In : *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2018, p. 586-595. DOI : 10.1109/CVPR.2018.00068.
- [17] Maria DZIUBA et al. *Image Vectorization : A Review*. 2023. arXiv : 2306.06441 [cs.CV].
- [18] Peter SELINGER. *Potrace : A Polygon-Based Tracing Algorithm*. Technical report. Sept. 2003. URL : <https://potrace.sourceforge.net/potrace.pdf>.
- [19] Pradyumna REDDY et al. “Im2Vec : Synthesizing Vector Graphics without Vector Supervision”. In : *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2021, p. 7342-7351.
- [20] Jia DENG et al. “ImageNet : A Large-Scale Hierarchical Image Database”. In : *2009 IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, 2009, p. 248-255. DOI : 10.1109/CVPR.2009.5206848.
- [21] Oriane SIMÉONI et al. “DINOv3”. In : (2025). DOI : 10.48550/arXiv.2508.10104. arXiv : 2508.10104 [cs.CV].